

Análise de Código: `airflow_dag_pipeline.py`

Apache Airflow - Extração de Dados

Disciplina de Extração de Dados

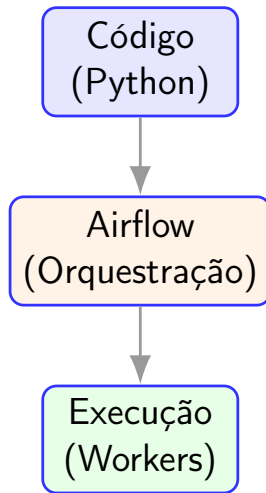
IBMEC

12 de maio de 2026

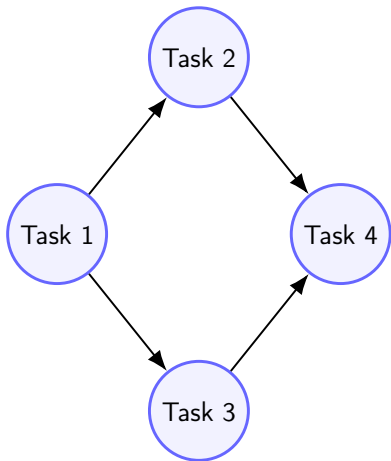
- Esta apresentação destrincha, linha por linha, o script `airflow_dag_pipeline.py`.
- O código constrói uma DAG (*Directed Acyclic Graph*) simples utilizando o Apache Airflow.
- Dividimos o script em **6 Blocos Principais** para facilitar o entendimento lógico e estrutural.

O que é o Apache Airflow?

- Plataforma *open-source* para criar, agendar e monitorar *workflows*.
- **Pipelines como código:** escritos em Python nativo.
- **Por que usar?**
 - Gerenciamento visual de dependências complexas.
 - Automação de *logs*, alertas e *retries*.
 - UI poderosa para monitoramento em tempo real.

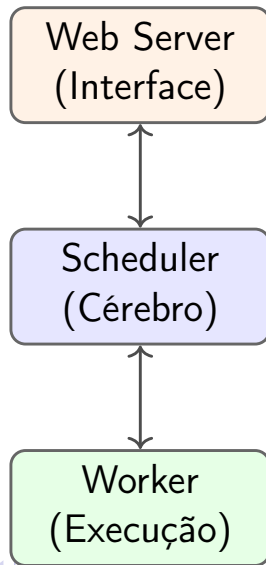


Principais Conceitos: DAGs e Tasks



- **DAG (*Directed Acyclic Graph*):**
 - Grafo Direcionado Acíclico (fluxo unidirecional, sem *loops*).
 - **Contêiner lógico:** orquestra a ordem e dependências.
- **Tasks (Tarefas):**
 - Nós do grafo: unidades atômicas de trabalho.
 - Ex: baixar dados, processar, salvar no banco.

- **Operators (Operadores):** Definem o que a Task faz.
 - `BashOperator`: executa *scripts* de terminal.
 - `PythonOperator`: funções em Python.
 - Conectores: AWS, GCP, Bancos, etc.
- **Arquitetura Básica:**
 - **Scheduler:** Agenda e dispara as tarefas.
 - **Worker:** Executa o trabalho na prática.
 - **Web Server:** *Dashboard* para o usuário.



Bloco 1: Importações Essenciais

```
4 from airflow import DAG
5 from airflow.operators.bash import BashOperator
6 from airflow.operators.python import PythonOperator
7 from datetime import datetime, timedelta
```

Explicação detalhada:

- **Linha 4:** Importa a classe DAG, fundamental para criar o contêiner lógico do pipeline.
- **Linha 5:** Importa o BashOperator, usado para executar comandos no terminal/sistema operacional (ex: scripts bash).
- **Linha 6:** Importa o PythonOperator, que permite encapsular e executar funções nativas de Python.
- **Linha 7:** Importa datetime e timedelta para manipulação de tempo e datas (essenciais para agendamento).

Bloco 2: Lógica de Execução

```
10 def print_hello():
11     """
12     Função simples em Python que será executada pelo Worker.
13     O retorno (opcional) será registrado nos Logs da interface gráfica.
14     """
15     mensagem = "Hello World, Airflow!"
16     print(mensagem)
17     return mensagem
```

Explicação detalhada:

- **Linhas 10-14:** Declaração de uma função Python convencional (`print_hello`) com uma *docstring* explicativa.
- **Linhas 15-17:** Cria uma string, imprime no console (registrado no log do Airflow) e a retorna. Retornos do `PythonOperator` podem ser passados para outras tasks via mecanismo `XCom`.

Bloco 3: Argumentos Padrão (Default Arguments)

```
21 default_args = {  
22     'owner': 'aluno_ibm8915',  
23     'retries': 1, # Tentativas automáticas em caso de falha  
24     'retry_delay': timedelta(minutes=5)  
25 }
```

Explicação detalhada:

- **Linha 21-25:** Dicionário com configurações base repassadas a todas as Tasks da DAG.
- **owner:** Define o "dono" do processo, útil para filtros na interface de usuário (UI).
- **retries:** Quantidade de vezes que a tarefa será reexecutada caso ocorra algum erro (falha).
- **retry_delay:** Define o intervalo de espera (5 minutos) antes de realizar a nova tentativa (usando o `timedelta` importado no Bloco 1).

Bloco 4: Instanciação da DAG (O Contêiner Lógico)

```
29 with DAG(  
30     dag_id='primeiro_hello_world_airflow',  
31     default_args=default_args,  
32     description='Pipeline de demonstração para a Aula 19',  
33     start_date=datetime(2026, 5, 1),  
34     schedule='@daily',  
35     catchup=False  
36 ) as dag:
```

Explicação detalhada:

- **Linha 29:** Uso do `with` (*Context Manager*) para associar tarefas à dag automaticamente.
- `dag_id` (**linha 30**): Identificador único e obrigatório da DAG na interface.
- `start_date` (**linha 33**): A data de início lógico do agendamento (*Cron*).
- `schedule` (**linha 34**) e `catchup` (**linha 35**): Frequência de execução (diária) e desativação do comportamento retroativo (*backfill*), impedindo de rodar passados.

Bloco 5: Construção das Tasks - BashOperator

```
40 # Task 1: Especializada em comandos do sistema operacional
41 tarefa_bash = BashOperator(
42     task_id='iniciando_pipeline',
43     bash_command='echo "Iniciando Pipeline..."',
44 )
```

Explicação detalhada:

- **Linha 41:** Instanciação de um operador `BashOperator`. Como está recuado dentro do `with`, pertence automaticamente a esta DAG.
- `task_id` (**linha 42**): Nome visual da tarefa na UI do Airflow. Precisa ser único dentro da DAG atual.
- `bash_command` (**linha 43**): Comando do Shell/Bash que será executado no *Worker* (neste caso, um `echo`).

Bloco 5: Construção das Tasks - PythonOperator

```
46 # Task 2: Especializada em rodar métodos nativos da linguagem
47 tarefa_python = PythonOperator(
48     task_id='imprimir_hello_world',
49     python_callable=print_hello
50 )
```

Explicação detalhada:

- **Linha 47:** Instanciação do `PythonOperator`. Ele não executa strings com código, mas sim referências a funções do escopo global.
- `task_id` (**linha 48**): Identificador único da task de Python.
- `python_callable` (**linha 49**): Aponta para a função `print_hello` (linhas 10-17). Note a ausência de parênteses `()`: apenas referenciamos o objeto da função, o executor do Airflow irá invocá-la depois.

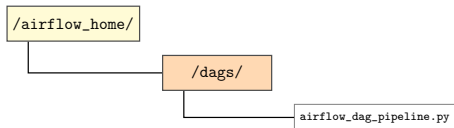
Bloco 6: Orquestração e Dependências Lógicas

```
52 # 6. Orquestração e Dependências Lógicas (Controlando o Fluxo)
53 # Garante que a tarefa Python só será iniciada após o sucesso absoluto
54 tarefa_bash >> tarefa_python
```

Explicação detalhada:

- **Linha 54:** Utiliza o operador de bitshift à direita (`>>`), sobrecarregado pelo Airflow para definir uma dependência (*Edge*).
- Significa que `tarefa_bash` é dependência direta da `tarefa_python`.
- Este fluxo sequencial garante que a `tarefa_python` **só** iniciará se, e somente se, a `tarefa_bash` for concluída com status de **sucesso**.

Pré-requisito: Onde salvar o código?



- Para que o *Scheduler* reconheça seu pipeline, o arquivo Python **não pode** ser salvo em qualquer lugar.
- **Instalação Local:** Salve o script dentro da pasta `dags` localizada no seu diretório base (geralmente `~/airflow/dags`).
- **Instalação via Docker:** Salve o script dentro da pasta `./dags` que está junto ao seu arquivo `docker-compose.yaml`.
- O Airflow escaneia essa pasta `dags` periodicamente (em *background*) para carregar novos pipelines.

Passo 0a: Inicialização (Instalação Local VirtualEnv)

```
$ airflow standalone
```

```
...
```

```
Airflow Webserver
```

```
Airflow Scheduler
```

- Antes de abrir o navegador, precisamos ligar os serviços do Airflow na sua máquina local ou servidor.
- Abra o terminal de comando (Linux/Mac/WSL) e digite o comando:

```
airflow standalone
```

- **Atenção:** Este comando inicia o Banco de Dados interno, o Web Server e o Scheduler em um único processo. Anote a **senha de admin** que aparecerá na tela e mantenha o terminal aberto!

Passo 0b: Inicialização (Instalação via Docker)

```
$ docker compose up -d
```

```
...
```

```
[+] Running 4/4
```

```
✓ Container airflow-postgres Started
```

```
✓ Container airflow-webser... Started
```

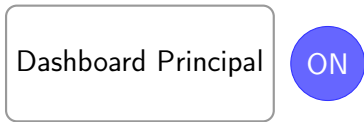
```
✓ Container airflow-schedu... Started
```

- Para quem instalou o Airflow usando a versão *containerizada* oficial.
- Abra o terminal na pasta raiz onde está o seu `docker-compose.yaml` e digite:

```
docker compose up -d
```
- A flag `-d` (*detached*) roda todos os serviços integrados em **background**.
- O login padrão criado pelo Docker costuma ser genérico: **Usuário:** airflow | **Senha:** airflow.

Passo 1: Encontrar e Ativar a DAG

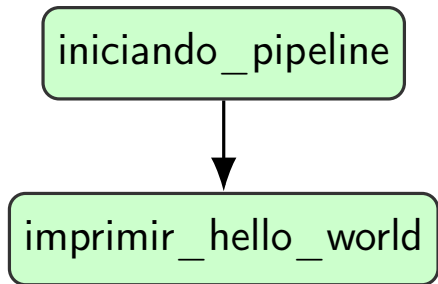
`primeiro_hello_world_airflow`



- Ao iniciar o ambiente (airflow webserver), acesse pelo navegador web (ex: localhost:8080).
- Procure pelo dag_id: `primeiro_hello_world_airflow`.
- **Ação:** Mova a chave seletora (*Toggle*) do lado esquerdo para **ON** (Ativo). Isso remove a DAG do estado de "pausa".

Passo 2: Disparar (Trigger) e Visualizar o Grafo

Status: Success



- **Disparo Manual:** Clique no botão de *Play* (▶) no canto superior direito e selecione **Trigger DAG**.
- **Acompanhamento:** Acesse a aba **Graph View** ou **Grid View**.
- Você verá as bordas das tarefas mudarem de cor indicando o status de transição até ficarem **verde escuro** (Sucesso).

Passo 3: Validando os Resultados nos Logs

```
[INFO] Executing...
```

```
Hello World, Airflow!
```

```
Marking success
```

- Como validar que a função Python funcionou?
- **Ação:** Clique na tarefa correspondente (`imprimir_hello_world`) que ficou verde.
- No menu lateral de detalhes, clique na aba **Log**.
- **Resultado:** Role a tela do terminal exibido na web e você verá a exata mensagem "Hello World, Airflow!" registrada pelo `print()`.

- **Modularidade:** O script ilustra de forma didática o padrão do Airflow: (1) importar bibliotecas, (2) definir parâmetros globais e lógicas, (3) declarar o DAG e (4) instanciar as tarefas (Operators).
- **Simplicidade Visual:** A operação de bitshift (») deixa a leitura do fluxo e a topologia de dependências clara no próprio código nativo.
- **Próximos Passos na Disciplina:** Após fixar este modelo em Python puro e Bash, podemos introduzir conexões com Bancos de Dados e Operadores específicos na Nuvem.